

QualiJournal admin-domap Cloud Run 배포 Workflow

QualiJournal의 `admin-domap` 서비스에 대한 CI/CD GitHub Actions 워크플로우 YAML을 작성했습니다. 이 워크플로우는 **main 브랜치로의 푸시** 이벤트를 트리거로 하여, 애플리케이션 컨테이너 이미지를 빌드하고 Google Cloud Run에 자동 배포합니다. **Workload Identity Federation** 기반 인증을 사용하여 GCP에 접근하며, 이를 위해 사용되는 서비스 계정에는 Cloud Run 관리 권한과 시크릿 접근 권한(예: `roles/run.admin`, `roles/secretmanager.secretAccessor`)이 부여되어야 합니다 ¹ ². 배포 후에는 **최신 리버전에 100% 트래픽을 전환**하고 서비스 헬스 체크 및 UI 요소 검증을 수행합니다. 모든 단계에서 오류 발생 시 워크플로우가 즉시 중단되고 (필요한 경우 Slack 알림을 전송하도록 설정할 수 있습니다).

워크플로우 주요 특징

- **트리거:** `main` 브랜치에 푸시하면 자동으로 실행됩니다.
- **인증:** GitHub Actions의 Workload Identity Federation(OIDC)으로 GCP 인증을 수행하며, 별도의 JSON 키 없이 안전하게 권한을 위임합니다 ³. (Workflow에 필요한 GCP IAM 역할: Cloud Run Admin, Secret Manager Accessor 등)
- **이미지 빌드:** 저장소의 Dockerfile을 이용해 **Cloud Build**로 컨테이너 이미지를 빌드하고 Artifact Registry/GCR에 푸시합니다.
- **Cloud Run 배포:** 빌드된 이미지를 Cloud Run에 배포하며, 배포 직후 최신 리버전에 **100% 트래픽을 할당**합니다 (`gcloud run services update-traffic --to-latest` 사용 ⁴).
- **헬스 체크:** 배포 완료 후 `/health`, `/api/status`, `/api/report` 엔드포인트에 HTTP 요청을 보내 응답 코드 200을 받는지 확인합니다.
- **UI 요소 검증:** 서비스 루트 (`/` 또는 `/index.html`)의 HTML을 내려받아 **특정 버튼(id="btn-token-clear")** 존재 여부를 검사합니다.
- **오류 처리 및 알림:** 어느 단계에서든 오류가 발생하면 워크플로우를 실패로 중단하며, (선택 사항으로) Slack 웹훅 알림을 보낼 수 있도록 단계가 준비되어 있습니다 (기본적으로 주석 처리되어 비활성화).

아래는 위 조건을 만족하는 **GitHub Actions 워크플로우 YAML**입니다. 각 job은 순차적으로 실행되도록 구성되었으며 (`needs` 키워드 사용), 중요한 부분마다 **방어적 로직**과 **주석**을 포함하여 오류를 방지하고 이해를 돕고 있습니다.

```
name: Deploy admin-domap to Cloud Run
```

```
on:
```

```
  push:
```

```
    branches: ['main'] # main 브랜치 푸시에 트리거
```

```
# 전역 환경 변수: gcloud 명령어에 사용할 프로젝트, 서비스, 리전 등의 설정
```

```
env:
```

```
  PROJECT_ID: ${ secrets.GCP_PROJECT_ID } # GCP 프로젝트 ID (GitHub Secrets에 저장)
```

```
  SERVICE_NAME: admin-domap # Cloud Run 서비스 이름
```

```
  REGION: asia-northeast3 # Cloud Run 배포 리전 (예: asia-northeast3 - 서울)
```

```
  IMAGE_URI: gcr.io/${ secrets.GCP_PROJECT_ID }/admin-domap:${ github.sha } # 컨테이너 이미지 경로 (커밋 해시 태그)
```

jobs:

1. Build Job: Docker 이미지 빌드 및 레지스트리에 푸시

build:

name: Build Container Image

runs-on: ubuntu-latest

Workload Identity Federation 사용을 위한 OIDC 토큰 권한 설정

permissions:

contents: read # 코드를 checkout하기 위한 권한

id-token: write # OIDC ID 토큰 발행 권한 (WIF 인증에 필요)

steps:

- name: Checkout source code

uses: actions/checkout@v4

- name: Authenticate to Google Cloud (WIF)

uses: google-github-actions/auth@v3

with:

project_id: \${{ env.PROJECT_ID }}

workload_identity_provider: \${{ secrets.GCP_WORKLOAD_IDENTITY_PROVIDER }}

service_account: \${{ secrets.GCP_SERVICE_ACCOUNT }}

위: GitHub OIDC 토큰을 통해 GCP 서비스 계정으로 액세스 (별도 키 불필요).

secrets.GCP_WORKLOAD_IDENTITY_PROVIDER 는 WIF 프로바이더 리소스 ID,

secrets.GCP_SERVICE_ACCOUNT 는 대상 서비스 계정 이메일이어야 합니다.

- name: Set up gcloud CLI

uses: google-github-actions/setup-gcloud@v1

with:

project_id: \${{ env.PROJECT_ID }}

install_components: beta # 필요한 경우 beta 컴포넌트 (Cloud Run 최신 기능용)

export_default_credentials: true

gcloud CLI 설치 및 인증 구성 (auth 액션의 자격증명을 ADC로 설정).

- name: Build and push image with Cloud Build

env:

Cloud Build에 필요한 환경변수 (프로젝트 ID 등) - 윗단계 env에서도 이미 설정됨

PROJECT_ID: \${{ env.PROJECT_ID }}

IMAGE_URI: \${{ env.IMAGE_URI }}

run: |

Cloud Build를 통해 Docker 이미지 빌드 및 레지스트리로 푸시

프로젝트의 Dockerfile을 기반으로 이미지 빌드

gcloud builds submit --project "\$PROJECT_ID" --tag "\$IMAGE_URI" .

if [\$? -ne 0]; then

echo " Cloud Build failed. Stopping workflow." >&2

exit 1

else

echo " Image built and pushed: \$IMAGE_URI"

fi

참고: Cloud Build API가 활성화되어 있어야 하며, 사용되는 서비스 계정에 Cloud Build 권한 필요.

(이미지 푸시는 \$IMAGE_URI 경로에 따라 GCR/Artifact Registry에 진행됩니다.)

2. Deploy Job: Cloud Run에 이미지 배포 및 트래픽 전환

deploy:

```

name: Deploy to Cloud Run
runs-on: ubuntu-latest
needs: [build] # build 잡 완료 후 실행
permissions:
  contents: read
  id-token: write # WIF 인증에 필요
steps:
  - name: Authenticate to Google Cloud (WIF)
    uses: google-github-actions/auth@v3
    with:
      project_id: ${{ env.PROJECT_ID }}
      workload_identity_provider: ${{ secrets.GCP_WORKLOAD_IDENTITY_PROVIDER }}
      service_account: ${{ secrets.GCP_SERVICE_ACCOUNT }}

  - name: Set up gcloud CLI
    uses: google-github-actions/setup-gcloud@v1
    with:
      project_id: ${{ env.PROJECT_ID }}
      install_components: beta
      export_default_credentials: true

  - name: Deploy to Cloud Run (no traffic)
    env:
      SERVICE_NAME: ${{ env.SERVICE_NAME }}
      REGION: ${{ env.REGION }}
      PROJECT_ID: ${{ env.PROJECT_ID }}
      IMAGE_URI: ${{ env.IMAGE_URI }}
    run: |
      set -e # 명령어 실패 시 즉시 종료
      echo " Deploying $SERVICE_NAME to Cloud Run..."
      # 최신 리비전 배포 (초기에는 트래픽 할당하지 않음)
      gcloud run deploy "$SERVICE_NAME" \
        --image "$IMAGE_URI" \
        --region "$REGION" \
        --project "$PROJECT_ID" \
        --platform managed \
        --no-traffic \
        --allow-unauthenticated
      echo " Deployment command executed for $SERVICE_NAME"
      # --no-traffic: 배포 후 바로 트래픽 전환을 하지 않음 (아래 단계에서 수동 전환).
      # --allow-unauthenticated: 인증되지 않은 접근 허용 (필요 시 조정).

  - name: Shift 100% traffic to new revision
    env:
      SERVICE_NAME: ${{ env.SERVICE_NAME }}
      REGION: ${{ env.REGION }}
      PROJECT_ID: ${{ env.PROJECT_ID }}
    run: |
      set -e
      echo " Shifting all traffic to the latest revision..."
      gcloud run services update-traffic "$SERVICE_NAME" \

```

```

--platform managed \
--region "$REGION" \
--project "$PROJECT_ID" \
--to-latest
echo " Traffic migrated to latest revision of $SERVICE_NAME"
# 위 명령은 Cloud Run 서비스의 최신 리비전에 100% 트래픽을 할당합니다 4 .

```

3. Test Job: 서비스 헬스 체크 및 UI 검증

test:

name: Post-deployment Tests

runs-on: ubuntu-latest

needs: [deploy] # deploy 잡 완료 후 실행

permissions:

contents: read

id-token: write # WIF 인증에 필요 (gcloud 사용을 위해)

steps:

- name: Authenticate to Google Cloud (WIF)

uses: google-github-actions/auth@v3

with:

project_id: \${{ env.PROJECT_ID }}

workload_identity_provider: \${{ secrets.GCP_WORKLOAD_IDENTITY_PROVIDER }}

service_account: \${{ secrets.GCP_SERVICE_ACCOUNT }}

- name: Set up gcloud CLI

uses: google-github-actions/setup-gcloud@v1

with:

project_id: \${{ env.PROJECT_ID }}

install_components: beta

export_default_credentials: true

- name: Retrieve Cloud Run Service URL

id: get-url

env:

SERVICE_NAME: \${{ env.SERVICE_NAME }}

REGION: \${{ env.REGION }}

PROJECT_ID: \${{ env.PROJECT_ID }}

run: |

Cloud Run 서비스의 URL 가져오기

SERVICE_URL=\$(gcloud run services describe "\$SERVICE_NAME" \

--platform managed --region "\$REGION" --project "\$PROJECT_ID" \

--format="value(status.url)")

echo "Service URL: \$SERVICE_URL"

다음 단계에서 사용하기 위해 환경 변수로 등록

echo "SERVICE_URL=\$SERVICE_URL" >> \$GITHUB_ENV

id: get-url 로 지정하여 필요시 outputs로도 참조 가능 (여기선 GITHUB_ENV로 설정).

- name: Health check endpoints (retry up to 5 times)

env:

SERVICE_URL: \${{ env.SERVICE_URL }}

run: |

set -e

```

# HEALTH_URLS 목록을 순회하며 각각 최대 5회까지 상태 체크
HEALTH_URLS="/health /api/status /api/report"
for endpoint in $HEALTH_URLS; do
    echo " Checking $endpoint..."
    success=false
    # 최대 5번 재시도 루프
    for attempt in {1..5}; do
        http_code=$(curl -o /dev/null -s -w "%{http_code}" "$SERVICE_URL$endpoint")
        if [ "$http_code" = "200" ]; then
            echo " $endpoint responded with 200 OK"
            success=true
            break
        else
            echo "⚠ $endpoint attempt $attempt returned $http_code (expected 200)"
            sleep 3 # 잠시 대기 후 재시도
        fi
    done
    # 만약 해당 endpoint에서 5번 시도 후에도 성공 못하면 실패 처리
    if [ "$success" = false ]; then
        echo " $endpoint did not return 200 OK after 5 attempts." >&2
        exit 1
    fi
done

```

- name: Verify UI element on homepage

env:

SERVICE_URL: \${ env.SERVICE_URL }

run: |

set -e

echo " Fetching homepage HTML for content check..."

HTML_CONTENT=\$(curl -L -s "\$SERVICE_URL") # 루트 경로의 HTML 응답

특정 버튼 요소(id="btn-token-clear") 존재 여부 검사

if echo "\$HTML_CONTENT" | grep -q 'id="btn-token-clear"'; then

echo " Found 'btn-token-clear' button on homepage."

else

echo " 'btn-token-clear' button NOT found on homepage!" >&2

exit 1

fi

(Optional) Slack Notification on Failure: 워크플로우 실패 시 Slack 알림 전송

- name: Notify Slack (on failure)

if: always() # 항상 실행하여 실패 여부를 판단

env:

JOB_STATUS: \${ job.status }

SLACK_WEBHOOK_URL: \${ secrets.SLACK_WEBHOOK_URL }

run: |

if ["\$JOB_STATUS" != "success"]; then

echo " Sending failure notification to Slack..."

Slack Webhook으로 실패 메시지 전송

curl -X POST -H 'Content-type: application/json' \

--data '{"text": "[Alert] admin-domap 배포 워크플로우 실패 : 확인 필요"}' \

```
# $SLACK_WEBHOOK_URL
# fi
# Slack 웹훅 URL은 GitHub Secrets에 저장하여 사용합니다. 이 단계는 기본적으로 비활성화되어 있으며,
# 필요 시 주석을 해제해 사용합니다.
```

구성 설명 및 유의사항

• **Workload Identity Federation 인증:** 각 Job의 초기 단계에서 `google-github-actions/auth@v3` 액션을 통해 OIDC 토큰으로 GCP에 인증합니다. 이 방식을 사용하면 JSON 키 없이도 GitHub Actions 워크플로우가 단계 자격증명을 얻어 GCP 리소스에 접근할 수 있습니다³. 이때 사용되는 GCP 서비스 계정에는 Cloud Run 배포와 시크릿 접근을 위한 IAM 역할 (예: Cloud Run Admin, Secret Manager Secret Accessor 등)이 사전에 할당되어 있어야 합니다¹. 또한 Workflow YAML의 `permissions: id-token: write` 설정을 통해 OIDC token 발급 권한을 부여해야 합니다 (GitHub OIDC 인증 필수 설정).

• **이미지 빌드 단계:** `build` Job에서는 먼저 코드를 체크아웃하고 (`actions/checkout@v4`), `gcloud CLI`를 설정한 다음 `gcloud builds submit` 명령으로 Cloud Build 서비스를 통해 Docker 이미지를 빌드합니다. `$IMAGE_URI` 환경 변수에 GCR/Artifact Registry 경로가 지정되어 있으며 (`gcr.io/$PROJECT_ID/admin-domap:$GITHUB_SHA` 형식), Cloud Build가 완료되면 해당 경로로 컨테이너 이미지가 푸시됩니다. **방어 로직:** 빌드 명령 뒤 `$?` 출력을 검사하여 (`if [ $? -ne 0 ]`) Cloud Build 실패 시 `exit 1`로 Job을 중단하도록 했습니다. (GitHub Actions에서는 Step 내 명령이 에러 코드를 반환하면 자동으로 그 Job이 실패하지만, 여러 명령을 나란히 사용하는 경우 `set -e`를 켜거나 위와 같은 수동 체크를 통해 다음 명령이 실행되지 않도록 합니다.)

• **배포 및 트래픽 전환:** `deploy` Job은 `needs: [build]` 의존관계로 빌드 성공 후 실행됩니다. 여기서 `gcloud run deploy` 명령으로 Cloud Run 서비스에 새 revision을 배포합니다. `--no-traffic` 옵션을 사용하여 배포 시 즉시 트래픽을 보내지 않도록 한 뒤, 다음 단계에서 `gcloud run services update-traffic --to-latest` 명령으로 **100% 트래픽을 최신 리비전으로 전환**합니다⁴. 이 2단계 절차는 기존 리비전에 트래픽이 남지 않도록 확실히 하고, 필요할 경우 배포 후 검증을 거친 뒤 트래픽을 옮기는 시나리오도 지원합니다. 배포 명령과 트래픽 전환 명령 모두 실패 시 즉시 Job이 실패하도록 `set -e`를 사용했습니다.

• **헬스 체크 및 UI 검증:** `test` Job은 배포 성공 후 Cloud Run 서비스의 동작을 검증합니다. 먼저 `gcloud run services describe`를 통해 **동적으로 서비스 URL**을 가져와 환경 변수로 저장합니다. 그 다음, 헬스스크립트를 사용하여 중요한 엔드포인트들을 순회하며 HTTP 상태 코드를 확인합니다. 각 엔드포인트에 최대 5번까지 재시도하도록 루프를 구성하여, 초기 배포 직후 컨테이너가 기동되는 지연 시간에도 테스트가 신뢰성 있게 통과될 확률을 높였습니다. 5회 시도 내에 200 응답을 받지 못하면 `exit 1`로 실패 처리하며, 어떤 엔드포인트에서든 실패가 발생하면 루프를 탈출하여 즉시 Job을 종료합니다. 또한 홈 페이지 HTML을 내려받아 (`curl -L -s`) **특정 버튼 요소** (`id="btn-token-clear"`) **존재 여부**를 `grep`으로 검색합니다. 해당 문자열이 없으면 `exit 1`로 실패 처리합니다. 이처럼 각 검증 단계에는 **명확한 실패 조건**과 메시지를 넣어 문제가 발생한 경우 어느 부분에서 실패했는지 로그로 파악하기 쉽게 했습니다.

• **에러 발생 시 동작:** 모든 Job에는 기본적으로 에러가 발생하면 그 Job과 이후 의존된 Job들이 중단됩니다. 위 구성에서 `build` 또는 `deploy` 단계가 실패하면 `test` Job은 실행되지 않습니다. 특히 `test` Job 내에는 여러 검증 단계가 있지만, `if [ ... ]; then exit 1; fi` 또는 `set -e` 사용으로 어느 하나라도 실패 시 즉시 그 Job 전체가 실패하도록 만들었습니다. 이런 흐름을 통해 **문제가 있는 배포는 프로덕션 트래픽에 반영되지 않고 조기에 감지**할 수 있습니다.

- **Slack 알림 (선택 사항):** 워크플로우 맨 마지막에 Slack 알림 단계를 예시로 포함했습니다. 이 단계는 기본적으로 주석 처리되어 실행되지 않으며, 필요시에만 주석을 해제하고 Slack 웹훅 URL을 설정해서 사용합니다. Slack 알림 단계는 `if: always()` 조건으로 정의되어, Job의 성공/실패 여부와 관계없이 항상 실행을 시도합니다. 내부 스크립트에서 `${{ job.status }}` 값을 확인하여 **Job이 실패한 경우에만** (`status != success`) 미리 구성된 Slack Incoming Webhook으로 메시지를 보내도록 구현했습니다. 이렇게 하면 워크플로우 실패 시에도 Slack 알림 단계가 스킵되지 않고 실행되어, 배포 실패를 실시간으로 통지할 수 있습니다. Webhook URL은 노출을 피하기 위해 GitHub Secrets (`SLACK_WEBHOOK_URL`)에 저장하여 참조하도록 했습니다. (실제 활용 시 팀의 Slack 채널에 맞게 메시지 내용이나 알림 조건을 조정하면 됩니다.)

이상으로 **QualiJournal** `admin-domap` 서비스를 위한 GitHub Actions 배포 워크플로우 YAML 구성과 각 단계에 대한 설명을 마칩니다. 이 Workflow를 사용하면 코드 변경을 main 브랜치에 푸시하는 것만으로 자동으로 컨테이너 빌드, Cloud Run 배포, 트래픽 전환, 배포 확인 및 (필요 시) 알림까지 한 번에 수행되어 배포 작업을 효율화할 수 있습니다. 각 설정 값(`PROJECT_ID`, `SERVICE_NAME`, `REGION` 등)은 자신의 환경에 맞게 수정하고, GCP IAM 권한과 Secret 등도 사전에 올바르게 준비해야 함을 유의하십시오.

1 3 Deploy to Cloud Run with GitHub Actions | Google Cloud Blog

<https://cloud.google.com/blog/products/devops-sre/deploy-to-cloud-run-with-github-actions/>

2 Deploy a secured serverless architecture using Cloud Run | Cloud Architecture Center | Google Cloud

<https://cloud.google.com/architecture/blueprints/serverless-blueprint>

4 Rollbacks, gradual rollouts, and traffic migration | Cloud Run | Google Cloud Documentation

<https://docs.cloud.google.com/run/docs/rollouts-rollbacks-traffic-migration>